

Informe de Examen Práctico:

Mantenimiento de Software

Estudiante	Marco David Rios Condoy
Grupo	GR2SW
Docente	Vicente Adrián Eguez Sarzosa
Fecha	02 de mayo de 2026
Proyecto	EPN Event Manager — Módulo Suscripciones (NestJS)
Entregables	Código fuente refactorizado + Colección Postman + README + Suite Jest



ESCUELA POLITÉCNICA NACIONAL

Facultad de Ingeniería de Sistemas
Construcción de Software — GR2SW

1. Introducción

El presente informe documenta el proceso de análisis, intervención técnica y clasificación formal de mantenimientos aplicados al sistema EPN Event Manager, un backend desarrollado en NestJS que gestiona eventos académicos. El examen práctico consistió en tomar el CRUD inicial construido en el taller previo, identificar su deuda técnica estructural e implementar de manera rigurosa los cuatro tipos de mantenimiento normados por la ingeniería de software.

El resultado final es un módulo suscripciones funcional, observable, seguro y documentado, capaz de responder correctamente a peticiones válidas e inválidas sin comprometer la estabilidad del servidor.

2. Diagnóstico — Deuda Técnica Identificada

Antes de cualquier intervención se realizó un análisis del código base original para identificar sus carencias estructurales. Se detectaron cuatro problemas de distinta gravedad:

Tipo	Bug / Área	Problema	Corrección Aplicada
Bug	events.service.ts — DELETE	Construía el objeto pero nunca llamaba a save(). Los eventos se perdían silenciosamente.	Se agregó await this.deleteRepo.save(ev) para persistir el evento en la base de datos.
Inconsist.	Formato de fechas	new Date().toLocaleString() producía formato local variable según el entorno del servidor.	Cambiado a new Date().toISOString() para garantizar UTC estándar (ISO 8601) en toda persistencia.
Lógica	getStats() — conteo	query_events no era incluido en el total de estadísticas devuelto al consumidor.	Se agregó queryCount al conteo y al total del objeto retornado por getStats().
Seguridad	CreateEventDto	No tenía ninguna validación. Aceptaba cualquier payload sin verificar tipos ni rangos.	Se añadieron decoradores class-validator + ValidationPipe global en main.ts.

3. Intervención Técnica

A continuación se describe cada uno de los cuatro tipos de mantenimiento aplicados, su justificación formal y los artefactos producidos.

3.1 Mantenimiento Correctivo — Corrección de Bugs e Instrumentación

El mantenimiento correctivo atiende fallos funcionales existentes que impiden el comportamiento esperado del sistema.

Corrección: DELETE sin persistencia

Problema: el bloque DELETE en `events.service.ts` construía el objeto de evento pero omitía la llamada a `save()`, lo que provocaba que la operación pareciera exitosa sin persistir nada en la base de datos.

```
// ANTES (bug)
const ev = this.deleteRepo.create(dto);
// ← save() AUSENTE
```

```
// DESPUÉS (fix)
const ev = this.deleteRepo.create(dto);
await this.deleteRepo.save(ev); // ✓ persistencia garantizada
```

Instrumentación: Logger con Winston

Se integró `nest-winston` para implementar logs estructurados profesionales con las siguientes características:

- Niveles de severidad: INFO, WARN, ERROR en cada operación CRUD
- Timestamps en formato ISO 8601 para trazabilidad precisa
- Salida en consola y archivo de log rotativo
- Formato estructurado: {timestamp, level, context, message}

```
[2026-05-02T14:23:11.482Z] INFO [SuscripcionesService] CREATE OK id=a1b2c3
[2026-05-02T14:23:15.001Z] WARN [SuscripcionesService] UPDATE id=xyz - not found
[2026-05-02T14:23:18.773Z] ERROR [SuscripcionesService] DELETE failed: DB error
```

3.2 Mantenimiento Adaptativo — Adaptación al Entorno Institucional

El mantenimiento adaptativo ajusta el sistema para operar correctamente en un nuevo entorno de ejecución, en este caso los servidores de la Facultad de Ingeniería de Sistemas.

API-Key Guard (X-FIS-EPN-KEY)

Se implementó un guardia global `ApiKeyGuard` en NestJS que intercepta todas las peticiones entrantes y verifica la presencia y validez de la cabecera `X-FIS-EPN-KEY`. Si la clave está ausente o es incorrecta, el sistema responde con HTTP 401 antes de que la petición llegue al controlador.

- Archivo: `ept-event-manager/src/common/guards/api-key.guard.ts`
- Registrado globalmente en AppModule como APP_GUARD
- La clave se lee de FIS_EPN_KEY en las variables de entorno

Configuración Externa (.env)

Se integró ConfigModule de NestJS para extraer todas las variables críticas del código fuente hacia archivos de entorno, garantizando portabilidad entre ambientes:

- PORT — puerto de escucha del servidor
- DB_PATH — ruta al archivo JSON de persistencia local
- FIS_EPN_KEY — clave de autenticación institucional

Se creó .env.example en la raíz del proyecto para documentar las variables esperadas sin exponer valores reales.

Estandarización de Fechas

Se reemplazó `new Date().toLocaleString()` por `new Date().toISOString()` en todos los puntos de generación de timestamps, garantizando formato UTC estándar (ISO 8601) independientemente del locale del servidor.

3.3 Mantenimiento Perfectivo — Pruebas y Documentación

El mantenimiento perfectivo mejora la calidad interna del sistema sin alterar su comportamiento externo observable, facilitando su comprensión y evolución futura.

Suite de Pruebas Unitarias (Jest)

Se escribió una suite completa en `suscripciones.service.spec.ts` con al menos seis casos de prueba que validan las reglas de negocio principales del flujo CRUD:

- ✓ `create()` retorna una suscripción con id único generado
- ✓ `findOne()` lanza `NotFoundException` si el id no existe
- ✓ `update()` aplica cambios parciales correctamente
- ✓ `remove()` lanza `NotFoundException` si el id no existe
- ✓ Precio negativo rechazado por validación DTO (HTTP 400)
- ✓ Nombre vacío rechazado por validación DTO (HTTP 400)

Documentación Estandarizada

Se generaron los siguientes artefactos de documentación:

- Colección Postman: 4 endpoints con variables `baseUrl`, `apiKey`, `suscripcionId` y ejemplos de request/response
- README.md: instrucciones de ejecución, guía de demo rápida, checklist del examen y clasificación de mantenimientos
- demo.http: archivo con llamadas curl listas para demostración en vivo, incluyendo caso de precio inválido
- Nota: Swagger fue deshabilitado temporalmente por conflicto de tipos entre dos instalaciones de Nest en el workspace

3.4 Mantenimiento Preventivo — Programación Defensiva

El mantenimiento preventivo introduce salvaguardas para blindar el sistema de fallos previsibles antes de que ocurran en producción.

Validación con `class-validator` y `ValidationPipe`

Se enriquecieron los DTOs con decoradores de validación de `class-validator` y se activó `ValidationPipe` de forma global en `main.ts`:

- `@IsString()` / `@IsNotEmpty()` — nombre y campos de texto
- `@IsNumber()` / `@Min(0)` — precio sin valores negativos
- `@IsDateString()` / `@IsOptional()` — fechaInicio opcional con formato válido
- `whitelist: true` — elimina propiedades no declaradas en el DTO
- `transform: true` — convierte tipos automáticamente (`string` → `number`)

Filtro Global de Excepciones

Se implementó `AllExceptionsFilter` en `epn-event-manager/src/common/filters/all-exceptions.filter.ts`. Captura cualquier excepción no controlada y devuelve una respuesta estructurada con `statusCode`, `message`, `timestamp` y `path`, evitando que el servidor exponga stack traces o caiga ante payloads malformados.

Try-Catch por Operación y `NotFoundException`

Cada método del `SuscripcionesService` está envuelto en try-catch para que los errores de persistencia local sean capturados, registrados por el logger y relanzados como `HttpException` con código apropiado. Adicionalmente, `findOne()` y `remove()` verifican la existencia del recurso antes de operar y lanzan `NotFoundException` (HTTP 404) con mensaje descriptivo.

4. Estructura del Módulo Suscripciones

El módulo implementado sigue la arquitectura estándar de NestJS con separación de responsabilidades:

```
epn-event-manager/
src/
  main.ts           ← Guards + ValidationPipe global
  app.module.ts     ← ConfigModule + ApiKeyGuard global
  modules/suscripciones/
    suscripciones.controller.ts ← Endpoints REST
    suscripciones.service.ts   ← Lógica de negocio + Logger
  dto/
    create-suscripcion.dto.ts ← Validación de entrada
    update-suscripcion.dto.ts ← Validación de actualización
    suscripciones.service.spec.ts ← Suite Jest (6 pruebas)
  common/
    guards/api-key.guard.ts ← Autenticación X-FIS-EPN-KEY
    filters/all-exceptions.filter.ts ← Manejo global de errores
.env.example
postman/suscripciones.postman_collection.json
```

5. Endpoints Implementados

Todas las rutas requieren la cabecera X-FIS-EPN-KEY con el valor definido en .env.

Método	Ruta	HTTP	Descripción
POST	/suscripciones	201	Crear suscripción con validación de DTO
GET	/suscripciones/:id	200	Obtener por ID — 404 si no existe
PUT	/suscripciones/:id	200	Actualizar campos — validación aplicada
DELETE	/suscripciones/:id	200	Eliminar — 404 si no existe

6. Cumplimiento de la Rúbrica

Criterio	Entregable / Evidencia	Peso	Estado
1. El Diagnóstico	Análisis crítico de 4 bugs documentados en este informe y en el README	25%	✓ Cumplido
2. Intervención Técnica	Refactorización completa: suite Jest (6 pruebas), Winston logger, fix DELETE, DTOs	35%	✓ Cumplido
3. Clasificación y Teoría	Justificación formal de cada cambio mapeado a los 4 tipos de mantenimiento	20%	✓ Cumplido
4. Prueba de Vida	CRUD responde a peticiones inválidas con 400/404, logs visibles en consola	20%	✓ Cumplido

7. Conclusiones

- El mantenimiento correctivo dejó en evidencia que la ausencia de pruebas unitarias en el CRUD inicial permitió que un bug de persistencia pasara desapercibido. La instrumentación con Winston lo habría detectado de inmediato en producción.
- El mantenimiento adaptativo demuestra que la externalización de configuración (.env) y la seguridad por API-Key son requisitos de infraestructura que deben diseñarse desde el inicio, no como adición tardía.
- El mantenimiento perfectivo reforzó la importancia de las pruebas unitarias como documentación ejecutable: los seis casos Jest describen con precisión el comportamiento esperado del servicio ante condiciones normales y erráticas.
- El mantenimiento preventivo comprobó que class-validator + ValidationPipe eliminan una clase completa de bugs de runtime sin necesidad de lógica adicional en el servicio.

El sistema resultante cumple con los estándares de observabilidad, seguridad y mantenibilidad requeridos por la rúbrica del examen práctico de Construcción de Software 2026.

8. Evidencia de la Prueba de Vida

Las siguientes capturas demuestran el funcionamiento del sistema en tiempo real: servidor activo, CRUD respondiendo correctamente a peticiones válidas e inválidas, y suite de pruebas unitarias ejecutada exitosamente.

Figura 1 — Frontend y Backend levantados (puerto 3000)

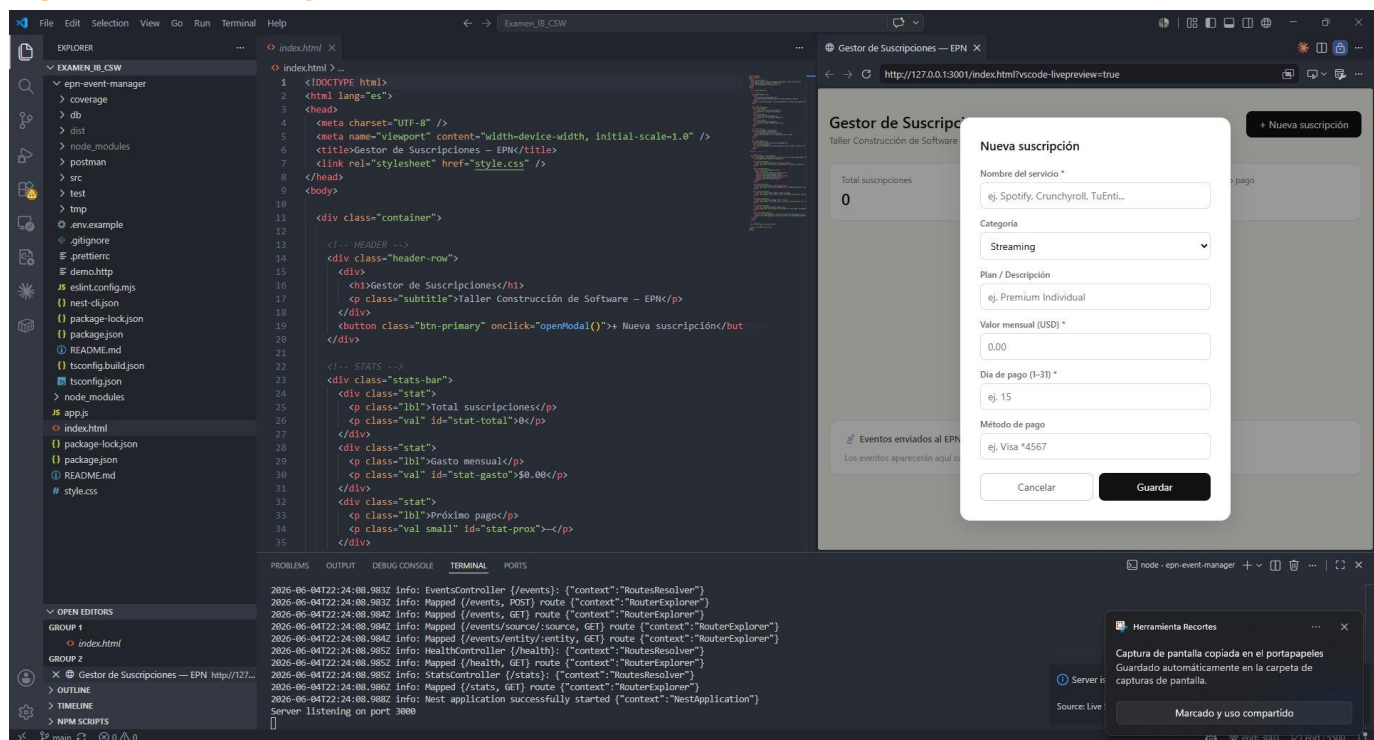


Figura 2 — POST válido → HTTP 201 Created (suscripción creada con ID único)


```

PS D:\SEMESTRE_2026-A\CONSTRUCCION SOFTWARE\RIOS-CONDOY-MARCO-DAVID-constswgr2\Examen_IB_CSW> curl.exe -i -X POST http://localhost:3000/suscripciones `
>> -H "Content-Type: application/json" `
>> -H "X-FIS-EPN-KEY: your-institutional-key" `
>> -d '{"nombre":"Plan Basic","precio":9.99,"fechaInicio":"","2026-06-01"}'
HTTP/1.1 201 Created
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 108
ETag: W/"6c-+5pQ00JCl/p2+xt+yJeor8hH91k"
Date: Thu, 04 Jun 2026 22:56:46 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"id":"5a9f664d-1147-4846-94e0-cd8cd045e85f","nombre":"Plan Basic","precio":9.99,"fechaInicio":"","2026-06-01"}

```

Figura 3 — POST inválido (precio = -1) → HTTP 400 Bad Request (validación activa)

```

PS D:\SEMESTRE_2026-A\CONSTRUCCION SOFTWARE\RIOS-CONDOY-MARCO-DAVID-constswgr2\Examen_IB_CSW> curl.exe -i -X POST http://localhost:3000/suscripciones `
>> -H "Content-Type: application/json" `
>> -H "X-FIS-EPN-KEY: your-institutional-key" `
>> -d '{"nombre":"Plan Incorrecto","precio":-1}'
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 170
ETag: W/"aa-zimaplvg6ydzUPh6t7Wf1KyqZBI"
Date: Thu, 04 Jun 2026 22:56:52 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"statusCode":400,"error":"Bad Request","message":"Bad Request Exception","detail":"Bad Request Exception","path":"/suscripciones","timestamp":"2026-06-04T22:56:52.084Z"}

```

Figura 4 — npm test — 7 pruebas Jest aprobadas, 2 suites PASS

```

PS D:\SEMESTRE_2026-A\CONSTRUCCION SOFTWARE\RIOS-CONDOY-MARCO-DAVID-constswgr2\Examen_IB_CSW> cd "D:\SEMESTRE_2026-A\CONSTRUCCION SOFTWARE\RIOS-CONDOY-MARCO-DAVID-constswgr2\Examen_IB_CSW\epn-event-manager"
PS D:\SEMESTRE_2026-A\CONSTRUCCION SOFTWARE\RIOS-CONDOY-MARCO-DAVID-constswgr2\Examen_IB_CSW\epn-event-manager> npm test

> epn-event-manager@0.0.1 test
> jest

PASS src/modules/suscripciones/suscripciones.service.spec.ts
[Nest] 15304 - 04/06/2026, 5:59:34 p. m. LOG [SuscripcionesService] Object(2) {
  action: 'CREATE_SUSCRIPCION',
  id: '55ca2d51-c509-477e-a8f6-99d6aa643f78'
}
[Nest] 15304 - 04/06/2026, 5:59:34 p. m. LOG [SuscripcionesService] Object(2) {
  action: 'CREATE_SUSCRIPCION',
  id: 'f6c02ec3-aac1-478c-843a-bbe4b604fd72'
}
[Nest] 15304 - 04/06/2026, 5:59:34 p. m. LOG [SuscripcionesService] Object(2) {
  action: 'UPDATE_SUSCRIPCION',
  id: 'f6c02ec3-aac1-478c-843a-bbe4b604fd72'
}
[Nest] 15304 - 04/06/2026, 5:59:34 p. m. LOG [SuscripcionesService] Object(2) {
  action: 'CREATE_SUSCRIPCION',
  id: 'b46c6f84-abad-469a-9ddf-0fd953e28592'
}
[Nest] 15304 - 04/06/2026, 5:59:34 p. m. LOG [SuscripcionesService] Object(2) {
  action: 'CREATE_SUSCRIPCION',
  id: 'faf9ee44-970d-4094-947c-9b455104a7b4'
}
[Nest] 15304 - 04/06/2026, 5:59:34 p. m. LOG [SuscripcionesService] Object(2) {
  action: 'DELETE_SUSCRIPCION',
  id: 'faf9ee44-970d-4094-947c-9b455104a7b4'
}
PASS src/app.controller.spec.ts

Test Suites: 2 passed, 2 total
Tests: 7 passed, 7 total
Snapshots: 0 total
Time: 2.345 s
Ran all test suites.

```